SYNOPSIS
ON
PhishEye: AI & ML based Cyberguard

Submitted in
Partial Fulfillment of the requirements for the Award of the Degree
of
Bachelor of Technology
In
Computer Science and Engineering
by

(Project Id: 26_CS_2L_19)

Hritik Kumar (2501640100427)
Kirt Raj Dixit (2501640100468)

Under the supervision of
Prasun Tripathi
(Assistant Professor)



Pranveer Singh Institute of Technology
Kanpur - Agra - Delhi National Highway - 19, Bhauti - Kanpur - 209305

1. Introduction

PhishEye (also titled PhishEye: AI & ML based Cyberguard) is a software-only cybersecurity system that scores phishing and lookalike-domain risk for a URL or pasted lure, explains the result in plain language, and can hold a Chrome tab closed until the scan finishes. The work belongs to applied cybersecurity: phishing detection, social-engineering analysis, and browser-side warning. The product form is a React web console, a FastAPI service, a SQLite store, and a Chrome Manifest V3 extension. There is no hardware, sensor, or IoT component.

Phishing is an impersonation attack in which a message or website mimics a trusted party (a bank, a delivery firm, or a government desk) so that the victim enters a password, an OTP, a UPI PIN, or card data. A valid TLS certificate (the browser padlock) does not prove that the site is the official brand. Lookalike domains, newly registered hosts, and shortened links are common technical tricks. Social engineering adds urgency, fear, authority, and credential pressure in the wording of the message.

PhishEye extracts URL and domain features (normalization, short-link unwrapping after an SSRF check, DNS existence, RDAP-style age, TLS facts, brand lookalike patterns), queries public threat-intelligence feeds, and applies rule-based phrase matching for psychological pressure (English and Hinglish). Those signals are combined by a documented decision path: confirmed feed hits apply score floors; multi-link content scans use static weighted fusion. The user sees a Simple View (OTP-language warning) and a Technical View (tags, reasons, provider match). Risk is reported as LOW (0–29), MEDIUM (30–49), HIGH (50–69), or CRITICAL (70–100).

Terms used here: URL lexical features (address structure, not the full page body); classifier means the coded decision procedure, not a shipped neural-network file; threat intelligence is an external feed lookup (a miss is not treated as safe); explainability is the Simple View plus Technical View. The repository has no trained scikit-learn, TensorFlow, or BERT model file. A fusion channel named ml copies the lexical URL score. Model confidence counts agreeing signals; it is not accuracy. No precision, recall, or F1 figure is claimed.

2. Project Objective

The project aims to give a non-technical user an understandable risk decision before credentials are typed, and to give an operator a console for history and reports. By the end of the mini-project the following are achieved:

1. Accept a URL, pasted SMS/WhatsApp/email text, a QR image, or a screenshot (QR always; optical character recognition only if Tesseract is installed on the host) and return a 0–100 risk score with a named level.

2. Analyse the address with lexical and domain heuristics (including lookalike/leetspeak, new-domain age, TLS inspection, and SSRF-safe short-link unwrapping).

3. Consult public threat-intelligence sources (OpenPhish, URLhaus, PhishTank, Phishing Army; Google Safe Browsing when an API key is configured) and apply confirmed-match floors so heuristics cannot veto a confirmed hit.

4. Detect social-engineering phrases (urgency, fear, account-blocking, impersonation, OTP/password requests) with rule-based matching, with a fuse cap so psychology alone cannot produce HIGH or CRITICAL.

5. Present a Simple View and a Technical View generated from the same decision.

6. Provide a Chrome extension that holds navigation, calls the same API, opens LOW destinations, asks on MEDIUM, and keeps HIGH, CRITICAL, or unavailable results closed.

7. Persist accounts and scan history in SQLite (users, url_checks, content_scans, extracted_links, user_reports, domain_reputation) with bcrypt password hashes and JWT-protected mutating routes.

8. Support community mark-as-scam/risky/safe as one signal, not as legal proof.

The end user is a student, a first-time digital-banking user, or any person who receives OTP/KYC lures and will not parse certificate jargon. The secondary user is the signed-in operator of the React console. The system does not read native phone SMS, Gmail, or WhatsApp inboxes; those channels are paste or visible-page links only.

3. Feasibility Study

The points below follow the stack that is already running, not a hypothetical redesign.

3.1 Technical Feasibility

The implementation uses Python 3.12.8 with FastAPI and Uvicorn for JSON APIs, the Python standard-library sqlite3 module for persistence, React 19 and Vite 8 for the console, and Chrome Manifest V3 for navigation intercept. OpenCV headless decodes QR codes from image bytes without fetching the encoded URL. Outbound HTTP clients (requests/httpx) call public feeds with timeouts. These components are mature, documented, and already integrated. SQLAlchemy model files exist in the repository but are not the live connection path; PostgreSQL is a future option, not the current database. A dedicated GPU is not required because there is no on-device neural-network training or inference graph.

3.2 Operational Feasibility

A user signs up, opens Scan, pastes a URL or message, or uploads a QR image, and reads Simple View first. The extension is installed with Chrome Developer mode (Load unpacked); it is not listed on the Chrome Web Store. If the API is unreachable, the extension treats the result as unavailable and does not open the destination. That operational rule matches the product goal: unknown is not safe. Daily use needs a running API (local Uvicorn or the Render service) and a modern Chromium browser.

3.3 Economic Feasibility

Core libraries are open source (FastAPI, React, Vite, OpenCV, NumPy, Pillow, pytest). The live database is a local SQLite file and does not require a paid database licence. Public feeds used in code (OpenPhish, URLhaus, PhishTank, Phishing Army) are queried as HTTP sources; optional Google Safe Browsing needs a key and is disabled when the key is absent. Hosting on Render free instances is sufficient for demonstration; the free API sleeps after idle time. No paid cloud GPU or commercial antivirus SDK is required for the current prototype. Exact rupee hosting cost is not measured in the repository. [VERIFY: current Render plan and any faculty-lab machine budget]

3.4 Schedule Feasibility

The codebase already contains the scanner, decision engines, psychology rules, extension gate, authentication, pytest suites, and a documented local start path. Remaining academic work is documentation, viva preparation, and optional hardening (retention jobs, a curated supervised model). That scope fits a B.Tech mini-project calendar when the implementation is treated as

complete and the write-up as the remaining deliverable. [VERIFY: official department start and end dates for session 2026–27]

3.5 Legal Feasibility

The system stores an account (username, email, password hash) and the URLs or scan metadata the user submits. It does not collect passwords, OTPs, or card numbers from web pages. The extension does not read password fields, cookies, or phone SMS. QR image bytes are decoded in memory and are not saved as files. Community reports are user-supplied labels, not legal verdicts. Scanning is performed on content the user chose to submit. Public threat feeds are used under their published access methods. The project does not claim a statutory cybersecurity certification. [VERIFY: institute ethics or data-handling form if the department requires one]

4. Methodology / Planning of Work

Work followed the live pipeline rather than a separate offline training notebook. There is no labelled production dataset file and no train/validation split in the product path.

Step 1. Problem framing: phishing as impersonation plus social engineering, not only a blacklist lookup.

Step 2. Input collection: URL form, pasted message, QR/screenshot upload, or Chrome navigation captured by the extension.

Step 3. Preprocessing: URL normalization (scheme, registered domain) and short-link unwrapping only after an SSRF deny-list (loopback, private ranges, link-local, cloud metadata 169.254.169.254).

Step 4. Feature extraction: lexical and host features, DNS existence, RDAP-style domain age (additional score if the domain is younger than 30 days), TLS inspection, brand/lookalike patterns, optional page-clutter signals.

Step 5. Threat-intelligence lookup: independent providers; a timeout is recorded as unavailable; a miss is no_match, not safe.

Step 6. Psychological analysis: lowercase substring / phrase rules with static points for urgency, fear, authority, coercion, financial pressure, credential/OTP requests, countdown, greed, secrecy, and emotional bait.

Step 7. Decision: a single URL uses max(heuristic, intel floor: 85 / 80 / 72 / 55). Content scans use static fusion weights that sum to 1.0 (url 0.22, domain 0.16, ml 0.10 as a URL-score copy, nlp 0.10, psychological 0.10, threat_intel 0.10, brand 0.08, redirect 0.04, sender 0.05, community 0.05). Psychology alone is capped at 35.

Step 8. Explanation: warnings.py builds Simple View and Technical View; results are stored in SQLite; the extension applies LOW / MEDIUM / HIGH-CRITICAL / unavailable gating.

Step 9. Verification: pytest modules under backend/tests/ cover API behaviour, psychology matches, intel status handling, QR, TLS helpers, URL normalize, and golden detection cases. These tests check behaviour. They are not a published accuracy study.

The diagrams below show the implemented architecture, data flow, use cases, and the live SQLite schema. PostgreSQL is omitted from the live figures.

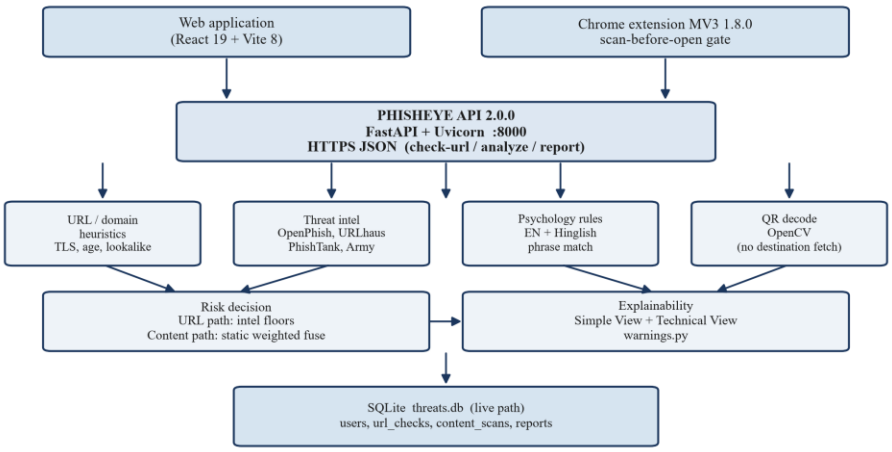


Fig. 1: System architecture of PhishEye (implemented path).

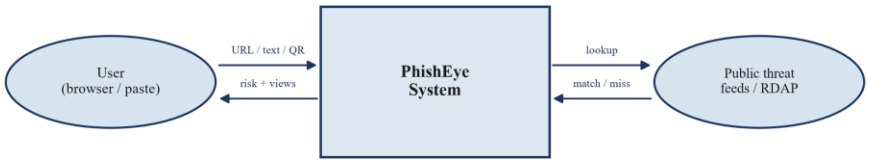


Fig. 2: Data-flow diagram, Level 0.

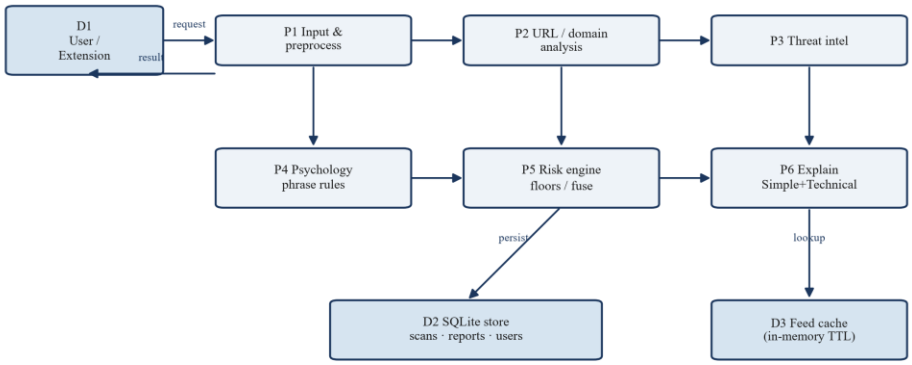


Fig. 3: Data-flow diagram, Level 1.

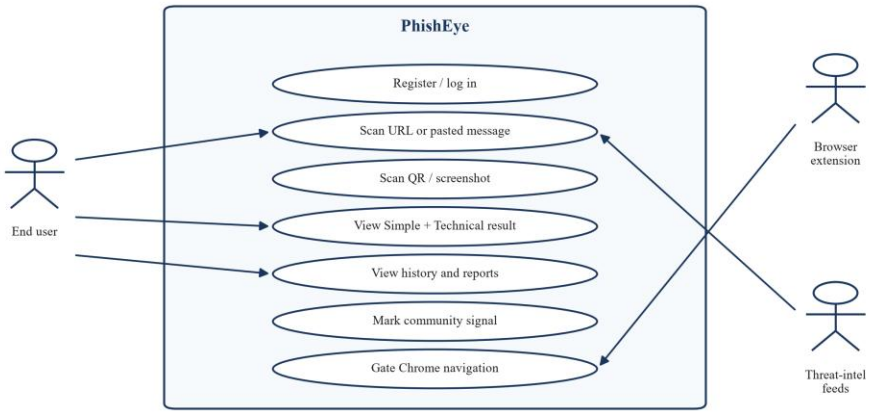


Fig. 4: Use-case diagram of PhishEye.

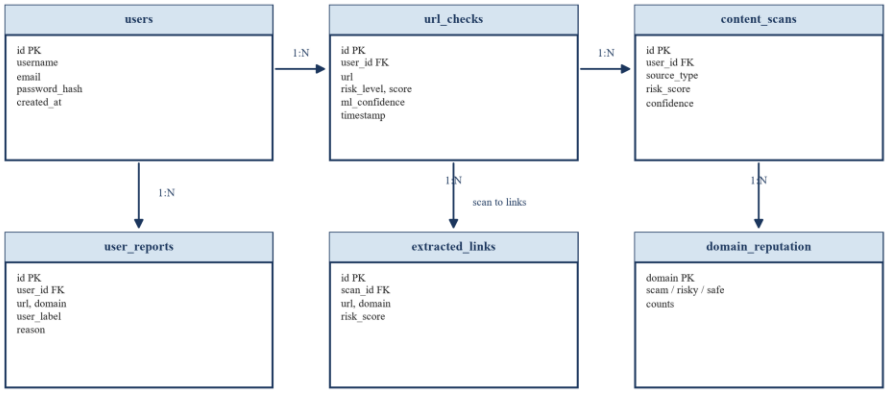


Fig. 5: Entity-relationship diagram of the live SQLite schema.

5. Tools / Technology Used

5.1 Minimum Hardware Requirements

The following minima are sufficient to run the FastAPI service, the Vite development server, Chrome, and OpenCV QR decode on a student laptop. A GPU is not used by the current detection path.

CPU: 64-bit dual-core processor, 2.0 GHz or higher (Intel Core i3 class or equivalent).

RAM: 8 GB minimum (16 GB recommended when Chrome, Node, and the API run together).

GPU: Not required. No trained deep-learning inference is shipped.

HDD / SSD: 10 GB free space for the repository, Python virtual environment, Node modules, and threats.db.

Others: Keyboard, display, and a working Internet connection for threat-feed lookups and optional Safe Browsing. A camera is not required; QR images are uploaded as files.

5.2 Minimum Software Requirements

Versions are taken from backend/runtime.txt, backend/requirements.txt, frontend/package.json, extension/manifest.json, and render.yaml. A lower-bound pin (for example fastapi>=0.115.0) is written as specified in the file.

Operating system: Windows 10/11 or modern Linux (Render deploy uses Linux).

Python / API: Python 3.12.8; FastAPI >=0.115.0; Uvicorn[standard] >=0.30.0; pydantic>=2.7.0; pydantic-settings>=2.2.0.

Auth / HTTP: passlib[bcrypt]>=1.7.4; bcrypt==4.0.1; python-jose[cryptography]>=3.3.0; requests>=2.31.0; httpx>=0.27.0; python-multipart>=0.0.9; python-dotenv>=1.0.0.

Vision: opencv-python-headless>=4.8.0; pillow>=10.0.0; numpy>=1.26.0 (image arrays for OpenCV, not a neural net).

Database / tests: sqlite3 (live). SQLAlchemy>=2.0.30 is listed but unused as the live path. pytest>=8.0.0. PostgreSQL is not connected.

Frontend: Node.js 22.12.0 on Render (render.yaml). React 19.2.8, react-dom 19.2.8, react-router-dom 7.18.2, axios 1.19.0, recharts 3.10.1, lucide-react 1.31.0, Vite 8.2.0, Tailwind CSS 4.3.3. [VERIFY: local Node version]

Extension / IDE: Chrome / Edge / Brave, Manifest V3, extension 1.8.0. Editor: Visual Studio Code or Cursor. [VERIFY: official lab IDE name]

Optional: Tesseract OCR (not in requirements.txt). Google Safe Browsing API key.

6. References

[1] S. Ramirez, "FastAPI documentation," [Online]. Available: https://fastapi.tiangolo.com/. [Accessed: 03-Oct-2026].

[2] Encode, "Uvicorn - An ASGI web server," [Online]. Available: https://www.uvicorn.org/. [Accessed: 03-Oct-2026].

[3] Pydantic, "Pydantic documentation," [Online]. Available: https://docs.pydantic.dev/. [Accessed: 03-Oct-2026].

[4] Meta / React Team, "React documentation," [Online]. Available: https://react.dev/. [Accessed: 03-Oct-2026].

[5] VoidZero / Vite Team, "Vite documentation," [Online]. Available: https://vite.dev/. [Accessed: 03-Oct-2026].

[6] Google, "Chrome Extensions (Manifest V3)," [Online]. Available: https://developer.chrome.com/docs/extensions/. [Accessed: 03-Oct-2026].

[7] SQLite Consortium, "SQLite documentation," [Online]. Available: https://www.sqlite.org/docs.html. [Accessed: 03-Oct-2026].

[8] OpenCV team, "OpenCV documentation," [Online]. Available: https://docs.opencv.org/. [Accessed: 03-Oct-2026].

[9] NumPy developers, "NumPy documentation," [Online]. Available: https://numpy.org/doc/. [Accessed: 03-Oct-2026].

[10] Pillow contributors, "Pillow documentation," [Online]. Available: https://pillow.readthedocs.io/. [Accessed: 03-Oct-2026].

[11] Python Software Foundation, "Python 3.12 documentation," [Online]. Available: https://docs.python.org/3.12/. [Accessed: 03-Oct-2026].

[12] OpenPhish, "OpenPhish phishing feed," [Online]. Available: https://openphish.com/. [Accessed: 03-Oct-2026].

[13] abuse.ch, "URLhaus API," [Online]. Available: https://urlhaus.abuse.ch/. [Accessed: 03-Oct-2026].

[14] PhishTank / Cisco Talos, "PhishTank," [Online]. Available: https://phishtank.org/. [Accessed: 03-Oct-2026].

[15] Phishing Army, "Phishing Army blocklist," [Online]. Available: https://phishing.army/. [Accessed: 03-Oct-2026].

[16] Google, "Safe Browsing API," [Online]. Available: https://developers.google.com/safe-browsing. [Accessed: 03-Oct-2026].

[17] pytest developers, "pytest documentation," [Online]. Available: https://docs.pytest.org/. [Accessed: 03-Oct-2026].